Introduction of Digital Electronics

1. What is Digital Electronics?

Digital Electronics is a branch of electronics and computer engineering focused on designing and implementing digital circuits, systems, and devices. These systems manipulate digital signals, where information is represented by binary values (0s and 1s). The discipline combines knowledge of logic theory, hardware components, and programming languages to create circuits that perform specific operations or tasks in a reliable and efficient way.

Digital Electronics encompasses the process of translating logical functions and complex problem-solving requirements into physical digital systems. This can range from simple circuits that control a small device, like an LED display, to complex systems like microprocessors and digital signal processors.

The steps typically involved in Digital Electronics are:

- **Requirement Analysis**: Understanding what the system needs to accomplish, its performance criteria, and constraints.
- **Conceptual Design**: Defining the logical operation of the system, often using Boolean algebra and truth tables.
- **Design and Synthesis**: Using logic gates (AND, OR, NOT, NAND, NOR, XOR) to build circuits that implement the desired operations. More advanced designs may use flip-flops, multiplexers, decoders, counters, and memory units.
- **Optimization**: Reducing the complexity of the design using techniques like Karnaugh Maps (K-Maps) and Quine–McCluskey methods to minimize the number of components needed, which also reduces power consumption and cost.
- **Verification and Testing**: Simulating the system in software to check if it performs as expected. After simulation, testing on hardware ensures reliability in real-world conditions.
- **Implementation**: Converting the design into a physical system, often using integrated circuits (ICs) or programmable devices like FPGAs (Field-Programmable Gate Arrays).

2. Why Digital Electronics Design?

Digital Electronics are the backbone of modern electronics due to several advantages over analog systems:

- **Accuracy and Reliability**: Digital Electronics use binary states (0 and 1), which makes them more robust to noise and interference compared to analog systems. This accuracy is critical in fields like computing, telecommunications, and medical devices.
- **Flexibility and Scalability**: Digital designs can be modified or scaled with relative ease, especially when programmable components like FPGAs or microcontrollers are used. This flexibility allows for rapid innovation and adaptation to new requirements.
- **High Performance**: With advanced design and fabrication technologies, digital systems can operate at very high speeds, enabling real-time processing and control in applications like video processing, signal processing, and telecommunications.

- **Integration with Other Systems**: Digital Electronics can be easily integrated with software, allowing for versatile applications in embedded systems, automation, and the Internet of Things (IoT).
- **Cost-effectiveness**: Digital components, particularly ICs, are highly standardized and cost-effective due to mass production. They also consume less power compared to equivalent analog systems, which is crucial in portable and battery-operated devices.

3. Where are Digital Electronics Used?

Digital Electronics plays a critical role in multiple fields. Some key areas of application include:

- **Computers and Microprocessors**: The core functionality of computers, such as processing and data storage, relies on digital systems. Microprocessors, the "brains" of computers, are complex digital systems built on digital design principles. Memory components like RAM and ROM are also designed digitally.
- **Communication Systems**: In telecommunications, digital designs are used in devices like modems, routers, and digital transmitters and receivers. Digital design enables encoding, decoding, compression, and transmission of data over networks reliably and efficiently.
- **Embedded Systems**: Embedded systems, which are computers dedicated to specific tasks within larger systems, rely heavily on digital design. These systems are found in applications ranging from household appliances (microwaves, washing machines) to automotive electronics (engine control units, anti-lock braking systems).
- **Consumer Electronics**: Almost all modern consumer electronics, such as smartphones, televisions, gaming consoles, and digital cameras, rely on digital systems for functionality. For instance, televisions use digital processing to handle video and audio signals, while smartphones combine various digital subsystems for communication, multimedia, and user interface functionality.
- **Control Systems and Industrial Automation**: Control systems in manufacturing and industrial processes use digital logic to monitor and control machinery. For example, Programmable Logic Controllers (PLCs) are digital systems programmed to automate industrial tasks.
- **Medical Devices**: Many diagnostic and therapeutic devices use digital designs for precision and reliability. For instance, MRI machines and CT scanners rely on digital signal processing to create detailed images, while infusion pumps use digital controls for precise drug delivery.
- **Military and Aerospace Applications**: In applications requiring high precision, reliability, and security, such as aircraft navigation, missile guidance, and satellite communications, digital systems are critical. They ensure that these systems can process data accurately and withstand harsh environments.

UNIT I: LOGIC SIMPLIFICATION AND COMBINATIONAL LOGIC DESIGN

Introduction: Logic simplification is a process used in digital circuit design to reduce the complexity of Boolean expressions. By applying Boolean algebra rules, redundant terms are eliminated, leading to a more efficient implementation of the circuit with fewer gates and components. This optimization is essential for both cost and power consumption and enhances overall performance.

Additionally, understanding different number systems, such as binary, octal, decimal, and hexadecimal, is crucial in digital design. These systems provide various ways to represent and manipulate data in digital circuits, with binary (base-2) being the most fundamental, as it directly corresponds to the on/off states of digital logic. Simplified logic expressions and number system conversions are foundational in creating combinational logic circuits, where the output depends solely on the current inputs, without any memory or feedback mechanisms.

NUMBER SYSTEMS

Decimal Number System

The number system that we use in our day-to-day life is the decimal number system. Decimal number system has base 10 as it uses 10 digits from 0 to 9. In decimal number system, the successive positions to the left of the decimal point represent units, tens, hundreds, thousands, and so on.

Each position represents a specific power of the base (10). For example, the decimal number 1234 consists of the digit 4 in the units position, 3 in the tens position, 2 in the hundreds position, and 1 in the thousands position. Its value can be written as

$(1 \times 1000) + (2 \times 100) + (3 \times 10) + (4 \times 1)$

$(1 \times 10^3) + (2 \times 10^2) + (3 \times 10^1) + (4 \times 10^0)$

$1000 + 200 + 30 + 4 = 1234$

Table 1.1: Some of the following number systems which are frequently used in computers

S.No.	Number System and Description
1	**Binary Number System** Base 2. Digits used : 0, 1
2	**Octal Number System** Base 8. Digits used : 0 to 7
3	**Hexa Decimal Number System** Base 16. Digits used: 0 to 9, Letters used : A- F

Binary Number System

Characteristics of the binary number system are as follows

- Uses two digits, 0 and 1
- Also called as base 2 number system
- Each position in a binary number represents a value of power of base (2) starting from the position numbered from 0,1,2,etc. Example 2^0,2^1,etc

Example:

Binary Number: 10101_2

Table 1.2: Calculating Decimal Equivalent

Step	Binary Number	Decimal Number
Step 1	10101_2	$((1 \times 2^4) + (0 \times 2^3) + (1 \times 2^2) + (0 \times 2^1) + (1 \times 2^0))_{10}$
Step 2	10101_2	$(16 + 0 + 4 + 0 + 1)_{10}$
Step 3	10101_2	21_{10}

Octal Number System

Characteristics of the octal number system are as follows

- Uses eight digits, 0,1,2,3,4,5,6,7
- Also called as base 8 number system
- Each position in a octal number represents a value of power of base (8) starting from the position numbered from 0,1,2,etc. Example 8^0,8^1,etc

Example:

Octal Number: 12570_8

Table 1.3: Calculating Decimal Equivalent

Step	Octal Number	Decimal Number
Step 1	12570_8	$((1 \times 8^4) + (2 \times 8^3) + (5 \times 8^2) + (7 \times 8^1) + (0 \times 8^0))_{10}$
Step 2	12570_8	$(4096 + 1024 + 320 + 56 + 0)_{10}$
Step 3	12570_8	5496_{10}

Hexadecimal Number System

Characteristics of hexadecimal number system are as follows

- Uses 10 digits and 6 letters, 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F
- Letters represent the numbers starting from 10. A = 10. B = 11, C = 12, D = 13, E = 14, F = 15
- Also called as base 16 number system
- Each position in a hexadecimal number represents a value of power of base (16) starting from the position numbered from 0,1,2,etc. Example 16^0,16^1,etc

Example:

Hexadecimal Number: $19FDE_{16}$

Table 1.4: Calculating Decimal Equivalent

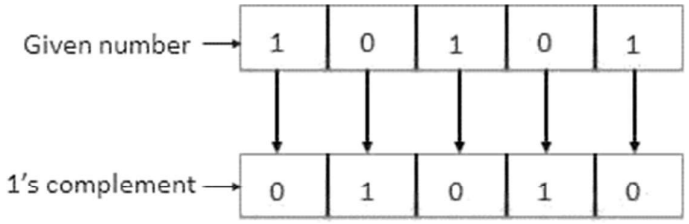
Step	Binary Number	Decimal Number
Step 1	$19FDE_{16}$	$((1 \times 16^4) + (9 \times 16^3) + (F \times 16^2) + (D \times 16^1) + (E \times 16^0))_{10}$
Step 2	$19FDE_{16}$	$((1 \times 16^4) + (9 \times 16^3) + (15 \times 16^2) + (13 \times 16^1) + (14 \times 16^0))_{10}$
Step 3	$19FDE_{16}$	$(65536 + 36864 + 3840 + 208 + 14)_{10}$
Step 4	$19FDE_{16}$	106462_{10}

Binary system complements

As the binary system has base r = 2. So the two types of complements for the binary system are 2's complement and 1's complement.

1's complement

The 1's complement of a number is found by changing all 1's to 0's and all 0's to 1's. This is called as taking complement or 1's complement. Example of 1's Complement is as follows.

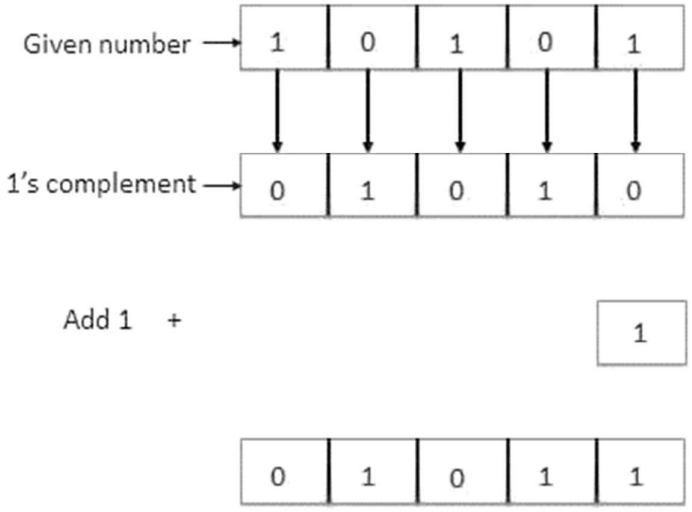


2's complement

The 2's complement of binary number is obtained by adding 1 to the Least Significant Bit (LSB) of 1's complement of the number.

2's complement = 1's complement + 1

Example of 2's Complement is as follows.



Binary arithmetic is essential part of all the digital computers and many other digital system.

Conversion

Other Base System to Decimal System

Step 1 Determine the column (positional) value of each digit (this depends on the position of the digit and the base of the number system).

Step 2 Multiply the obtained column values (in Step 1) by the digits in the corresponding columns.

Step 3 Sum the products calculated in Step 2. The total is the equivalent value in decimal.

Example

Binary Number: 11101_2

Table 1.5: Calculating Decimal Equivalent

Step	Binary Number	Decimal Number
Step 1	11101_2	$((1 \times 2^4) + (1 \times 2^3) + (1 \times 2^2) + (0 \times 2^1) + (1 \times 2^0))_{10}$

Step 2	11101_2	$(16 + 8 + 4 + 0 + 1)_{10}$
Step 3	11101_2	29_{10}

Binary Number: 11101_2 = Decimal Number: 29_{10}

Other Base System to Non-Decimal System

Step 1 Convert the original number to a decimal number (base 10).

Step 2 Convert the decimal number so obtained to the new base number.

Example

Octal Number : 25_8

Calculating Binary Equivalent

Step 1 - Convert to Decimal

Step	Octal Number	Decimal Number
Step 1	25_8	$((2 \times 8^1) + (5 \times 8^0))_{10}$
Step 2	25_8	$(16 + 5)_{10}$
Step 3	25_8	21_{10}

Octal Number : 25_8 = Decimal Number : 21_{10}

Step 2 - Convert Decimal to Binary

Step	Operation	Result	Remainder
Step 1	21 / 2	10	1
Step 2	10 / 2	5	0
Step 3	5 / 2	2	1
Step 4	2 / 2	1	0
Step 5	1 / 2	0	1

Decimal Number: 21_{10} = Binary Number : 10101_2

Octal Number: 25_8 = Binary Number : 10101_2

Shortcut Method ─ Binary to Octal

Step 1 Divide the binary digits into groups of three (starting from the right).

Step 2 Convert each group of three binary digits to one octal digit.

Example: **Binary Number : 10101_2**

Table 1.6: Calculating Octal Equivalent

Step	Binary Number	Octal Number
Step 1	10101_2	010 101
Step 2	10101_2	2_8 5_8
Step 3	10101_2	25_8

Binary Number : 10101_2 = Octal Number : 25_8

Shortcut Method ─ Octal to Binary

Step 1 Convert each octal digit to a 3-digit binary number (the octal digits may be treated as decimal for this conversion).

Step 2 Combine all the resulting binary groups (of 3 digits each) into a single binary number.

Example: Octal Number: 25_8

Table 1.7: Calculating Binary Equivalent

Step	Octal Number	Binary Number
Step 1	25_8	2_{10} 5_{10}
Step 2	25_8	010_2 101_2
Step 3	25_8	010101_2

Octal Number : 25_8 = Binary Number : 10101_2

Shortcut Method ─ Binary to Hexadecimal

Step 1 Divide the binary digits into groups of four (starting from the right).

Step 2 Convert each group of four binary digits to one hexadecimal symbol.

Example: Binary Number: 10101_2

Table 1.8: Calculating hexadecimal Equivalent

Step	Binary Number	Hexadecimal Number

Step 1	10101_2	0001 0101

Step 2	10101_2	1_{10} 5_{10}
Step 3	10101_2	15_{16}

Binary Number : 10101_2 = Hexadecimal Number : 15_{16}

Shortcut Method - Hexadecimal to Binary

Step 1 Convert each hexadecimal digit to a 4-digit binary number (the hexadecimal digits may be treated as decimal for this conversion).

Step 2 Combine all the resulting binary groups (of 4 digits each) into a single binary number.

Example: Hexadecimal Number : 15_{16}

Table 1.9: Calculating Binary Equivalent

Step	Hexadecimal Number	Binary Number
Step 1	15_{16}	1_{10} 5_{10}
Step 2	15_{16}	0001_2 0101_2
Step 3	15_{16}	00010101_2

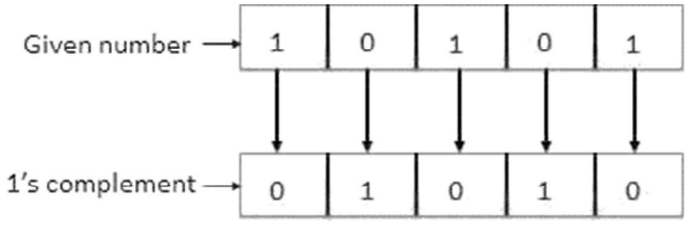
Hexadecimal Number : 15_{16} = Binary Number : 10101_2

Binary system complements

As the binary system has base r = 2. So the two types of complements for the binary system are 2's complement and 1's complement.

1's complement

The 1's complement of a number is found by changing all 1's to 0's and all 0's to 1's. This is called as taking complement or 1's complement. Example of 1's Complement is as follows.



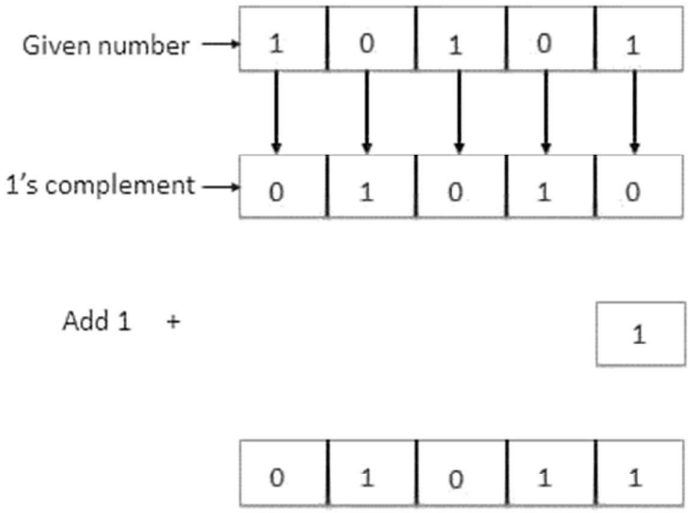
2's complement

The 2's complement of binary number is obtained by adding 1 to the Least Significant Bit

(LSB) of 1's complement of the number.

2's complement = 1's complement + 1

Example of 2's Complement is as follows.



Binary arithmetic is essential part of all the digital computers and many other digital system

BINARY CODES

Binary Coded Decimal (BCD) code

In this code each decimal digit is represented by a 4-bit binary number. BCD is a way to express each of the decimal digits with a binary code. In the BCD, with four bits we can represent sixteen numbers (0000 to 1111). But in BCD code only first ten of these are used (0000 to 1001). The remaining six code combinations i.e. 1010 to 1111 are invalid in BCD.

Decimal	0	1	2	3	4	5	6	7	8	9
BCD	0000	0001	0010	0011	0100	0101	0110	0111	1000	1001

Advantages of BCD Codes

- It is very similar to decimal system.
- We need to remember binary equivalent of decimal numbers 0 to 9 only.

Disadvantages of BCD Codes

- The addition and subtraction of BCD have different rules.
- The BCD arithmetic is little more complicated.
- BCD needs more number of bits than binary to represent the decimal number. So BCD is less efficient than binary.

Excess-3 code

The Excess-3 code is also called as XS-3 code. It is non-weighted code used to express decimal numbers. The Excess-3 code words are derived from the 8421 BCD code words adding (0011)2 or (3)10 to each code word in 8421. The excess-3 codes are obtained as follows

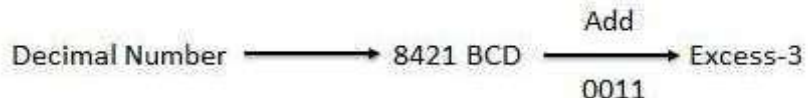


Table 1.10: BCD to Excess-3 Code

Decimal	BCD 8 4 2 1	Excess-3 BCD + 0011
0	0 0 0 0	0 0 1 1
1	0 0 0 1	0 1 0 0
2	0 0 1 0	0 1 0 1
3	0 0 1 1	0 1 1 0
4	0 1 0 0	0 1 1 1
5	0 1 0 1	1 0 0 0
6	0 1 1 0	1 0 0 1
7	0 1 1 1	1 0 1 0
8	1 0 0 0	1 0 1 1
9	1 0 0 1	1 1 0 0

Gray Code

It is the non-weighted code and it is not arithmetic codes. That means there are no specific weights assigned to the bit position. It has a very special feature that, only one bit will change each time the decimal number is incremented as shown in fig. As only one bit changes at a

time, the gray code is called as a unit distance code. The gray code is a cyclic code. Gray code cannot be used for arithmetic operation.

Table 1.11: BCD to Gray code

Decimal	BCD	Gray
0	0 0 0 0	0 0 0 0
1	0 0 0 1	0 0 0 1
2	0 0 1 0	0 0 1 1
3	0 0 1 1	0 0 1 0
4	0 1 0 0	0 1 1 0
5	0 1 0 1	0 1 1 1
6	0 1 1 0	0 1 0 1
7	0 1 1 1	0 1 0 0
8	1 0 0 0	1 1 0 0
9	1 0 0 1	1 1 0 1

Application of Gray code

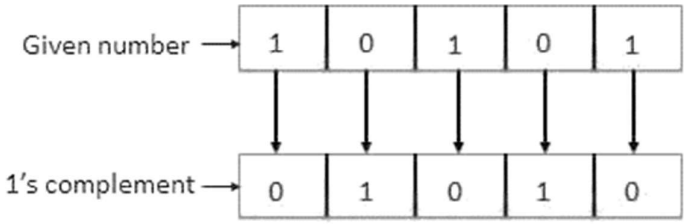
- Gray code is popularly used in the shaft position encoders.
- A shaft position encoder produces a code word which represents the angular position of the shaft.

Binary system complements

As the binary system has base r = 2. So the two types of complements for the binary system are 2's complement and 1's complement.

1's complement

The 1's complement of a number is found by changing all 1's to 0's and all 0's to 1's. This is called as taking complement or 1's complement. Example of 1's Complement is as follows.

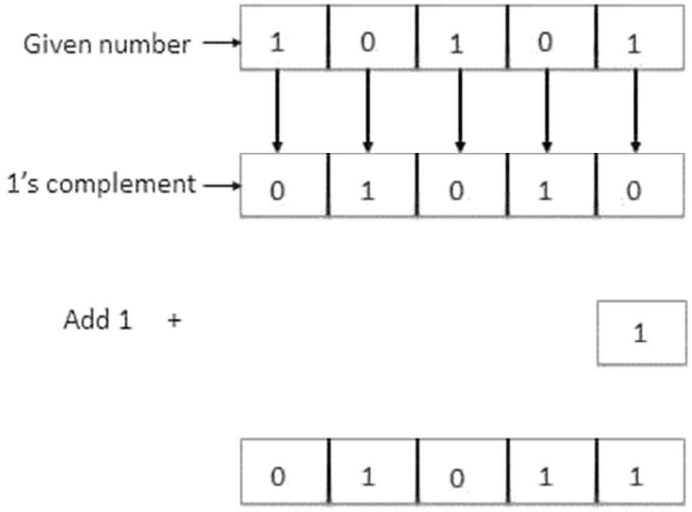


2's complement

The 2's complement of binary number is obtained by adding 1 to the Least Significant Bit (LSB) of 1's complement of the number.

2's complement = 1's complement + 1

Example of 2's Complement is as follows.



Binary arithmetic is essential part of all the digital computers and many other digital system.

Binary Addition

It is a key for binary subtraction, multiplication, division. There are four rules of binary addition.

Table 1.12: Binary addition

Case	A + B	Sum	Carry
1	0 + 0	0	0
2	0 + 1	1	0
3	1 + 0	1	0
4	1 + 1	0	1

In fourth case, a binary addition is creating a sum of (1 + 1 = 10) i.e. 0 is written in the given column and a carry of 1 over to the next column.

Example Addition

0011010 + 001100 = 00100110

```
   1 1          carry
 0 0 1 1 0 1 0  = 26₁₀
+0 0 0 1 1 0 0  = 12₁₀
---------------
 0 1 0 0 1 1 0  = 38₁₀
```

Binary Subtraction

Subtraction and Borrow, these two words will be used very frequently for the binary subtraction. There are four rules of binary subtraction.

Table 1.13: Binary subtraction

Case	A - B	Subtract	Borrow
1	0 - 0	0	0
2	1 - 0	1	0
3	1 - 1	0	0
4	0 - 1	0	1

Example Subtraction

0011010 - 001100 = 00001110

```
      1 1      borrow
  0 0 1 1 0 1 0  = 26₁₀
- 0 0 0 1 1 0 0  = 12₁₀
-----------------
  0 0 0 1 1 1 0  = 14₁₀
```

Binary Multiplication

Binary multiplication is similar to decimal multiplication. It is simpler than decimal multiplication because only 0s and 1s are involved. There are four rules of binary multiplication.

Table 1.13: Binary Multiplication

Case	A x B	Multiplication
1	0 x 0	0
2	0 x 1	0
3	1 x 0	0
4	1 x 1	1

Example:

0011010 x 001100 = 100111000

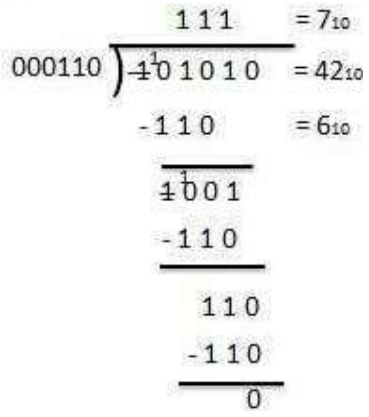
```
    0 0 1 1 0 1 0  = 26₁₀
  x 0 0 0 1 1 0 0  = 12₁₀
-------------------
    0 0 0 0 0 0 0
  0 0 0 0 0 0 0
0 0 1 1 0 1 0
0 0 1 1 0 1 0
-------------------
1 0 0 1 1 1 0 0 0  = 312₁₀
```

Binary Division

Binary division is similar to decimal division. It is called as the long division procedure.

Example Division

101010 / 000110 = 000111



Positive and Negative logic:

Binary logic is always 0s and 1s. There is only one little problem though, electricity doesn't come in 0s or 1s it comes in high or low voltage. So, you need to kind of assign which is which. And most of the time high voltage was assigned to be 1 and low voltage to be 0.

But you could also assign a high voltage to mean 0 and the low voltage to be 1 and there you have it: negative logic.

You can of course create whole circuits using negativ logic, but most of the time it's only used for things like enable inputs or similar. You can easily spot them by the line on top of the input name like: EN which would be an enable input that enables whatever can be enabled and disabled by that input with a low voltage on this input.